



NEW YORK UNIVERSITY

Modules & Architectures

Yann LeCun

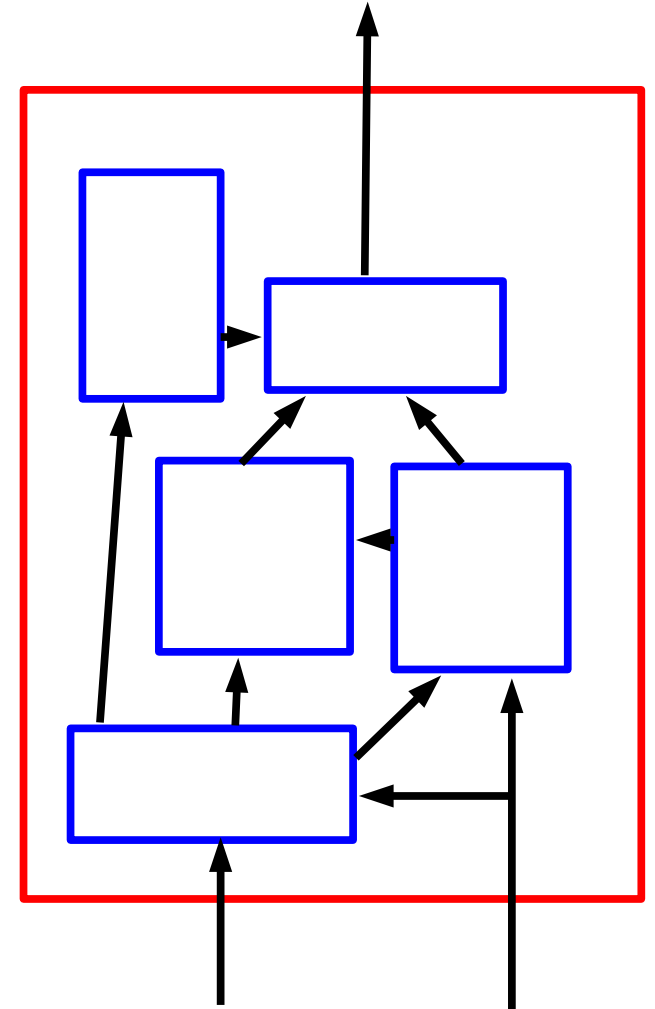
NYU - Courant Institute & Center for Data Science

Facebook AI Research

Deep Learning, NYU Spring 2021

Deep Learning: Assembling differentiable modules

- ▶ **Deep Learning Definition:**
 - ▶ Assembling networks of differentiable modules and training them with a gradient-based optimization method.
- ▶ **Any graph of modules is fine**
 - ▶ As long as there exist a partial order on the modules, and we can backpropagate gradient with respect to the relevant variables
- ▶ **If the graph has loops...**
 - ▶ we need to “unroll” them.
 - ▶ Recurrent networks and bakprop through time



Simple Neural Net

► Object-oriented version

- Uses predefined nn.Linear class, (which includes a bias vector)
- Uses torch.relu function
- State variables are temporary

```
import torch
```

```
from torch import nn
```

```
image = torch.randn(3, 10, 20)
```

```
d0 = image.nelement()
```

```
class mynet(nn.Module):
```

```
    def __init__(self, d0,d1,d2,d3):
```

```
        super().__init__()
```

```
        self.m0 = nn.Linear(d0, d1)
```

```
        self.m1 = nn.Linear(d1, d2)
```

```
        self.m2 = nn.Linear(d2, d3)
```

```
    def forward(self, x):
```

```
        z0 = x.view(-1)  ## flatten input tensor
```

```
        s1 = self.m0(x)
```

```
        z1 = torch.relu(s1)
```

```
        s2 = self.m1(z1)
```

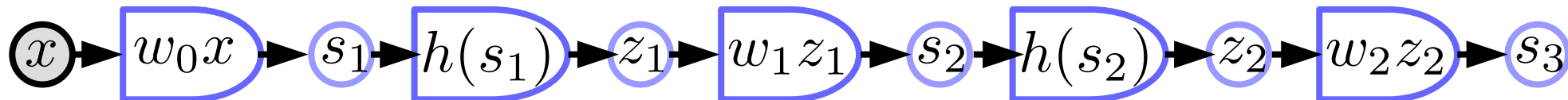
```
        z2 = torch.relu(s2)
```

```
        s3 = self.m2(z2)
```

```
        return s3
```

```
model = mynet(d0,60,40,10)
```

```
out = model(image)
```

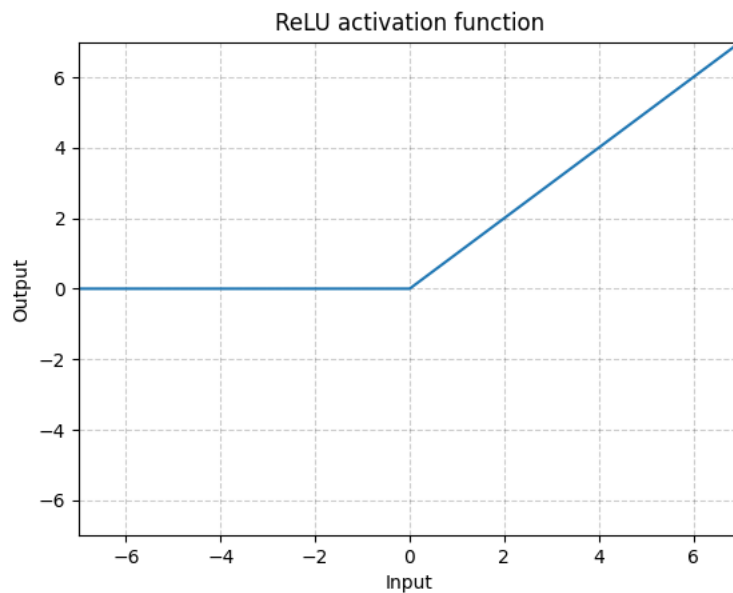


Activation Functions

In PyTorch & how to use them

ReLU – `nn.ReLU()`

$$\text{ReLU}(x) = (x)^+ = \max(0, x)$$



RReLU – nn.ReLU()

$$\text{RReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ ax & \text{otherwise} \end{cases}$$

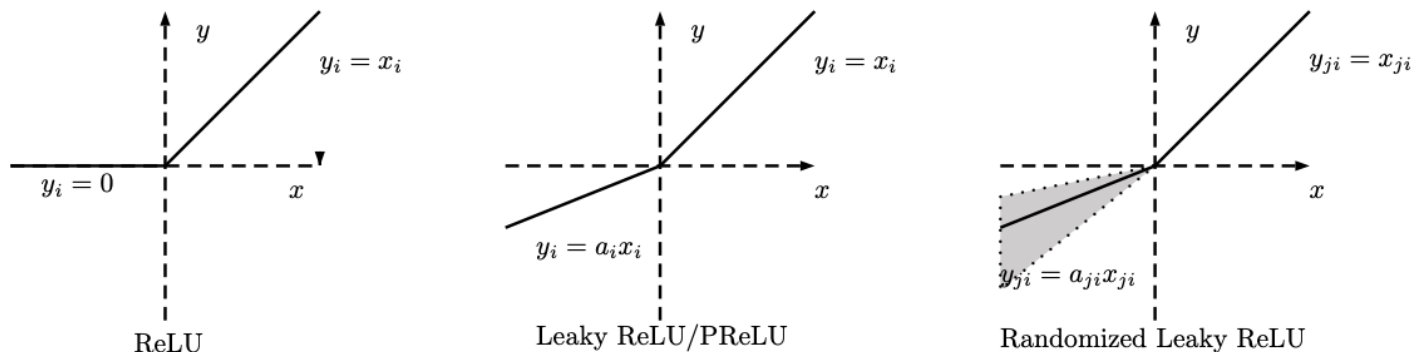
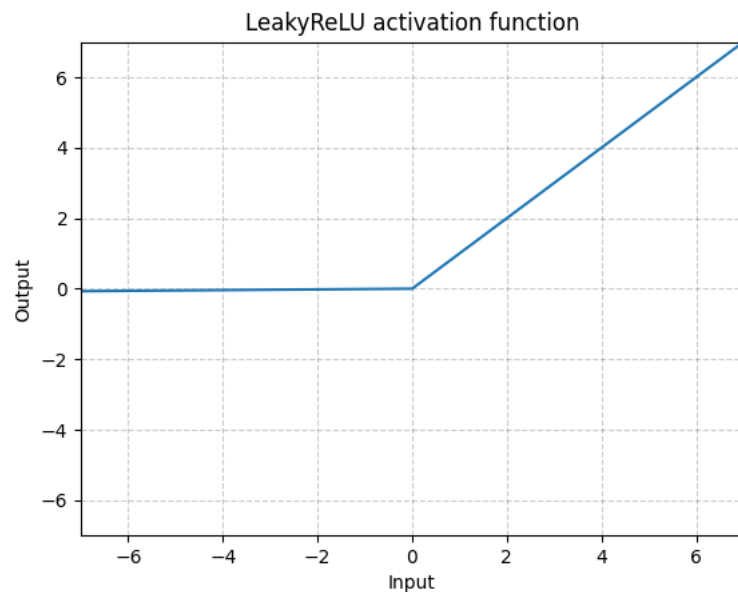


Figure 1: ReLU, Leaky ReLU, PReLU and RReLU. For PReLU, a_i is learned and for Leaky ReLU a_i is fixed. For RReLU, a_{ji} is a random variable keeps sampling in a given range, and remains fixed in testing.

LeakyReLU – `nn.LeakyReLU()`

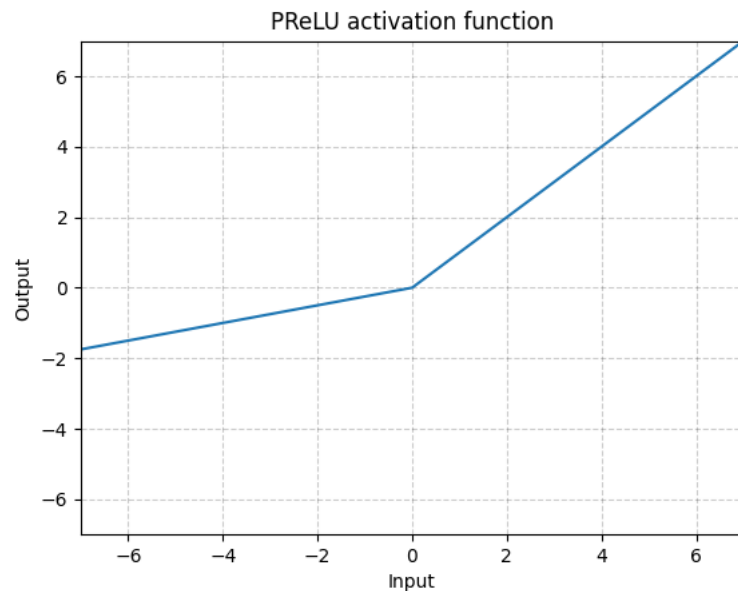
$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \text{negative_slope} \times x, & \text{otherwise} \end{cases}$$



PReLU – `nn.PReLU()`

$$\text{PReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{otherwise} \end{cases}$$

Here a is a learnable parameter. When called without arguments, `nn.PReLU()` uses a single parameter a across all input channels. If called with `nn.PReLU(nChannels)`, a separate a is used for each input channel.

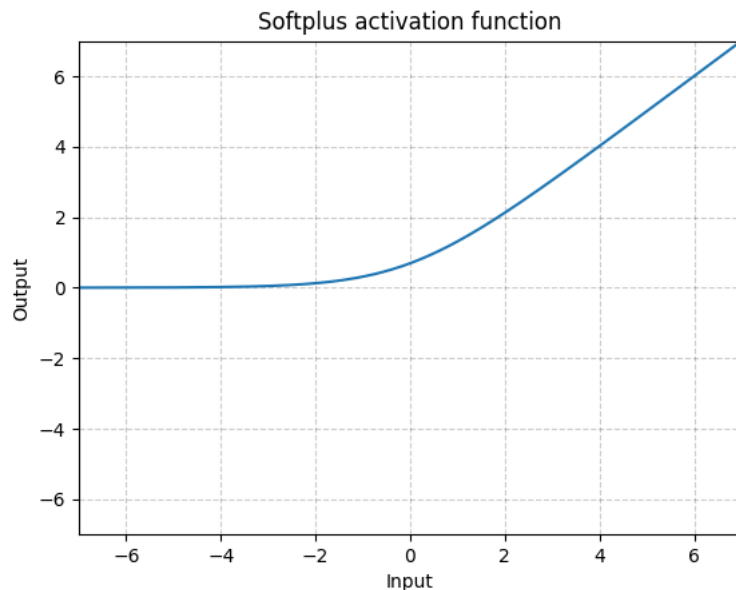


Softplus – `nn.Softplus()`

$$\text{Softplus}(x) = \frac{1}{\beta} * \log(1 + \exp(\beta * x))$$

SoftPlus is a smooth approximation to the ReLU function and can be used to constrain the output of a machine to always be positive.

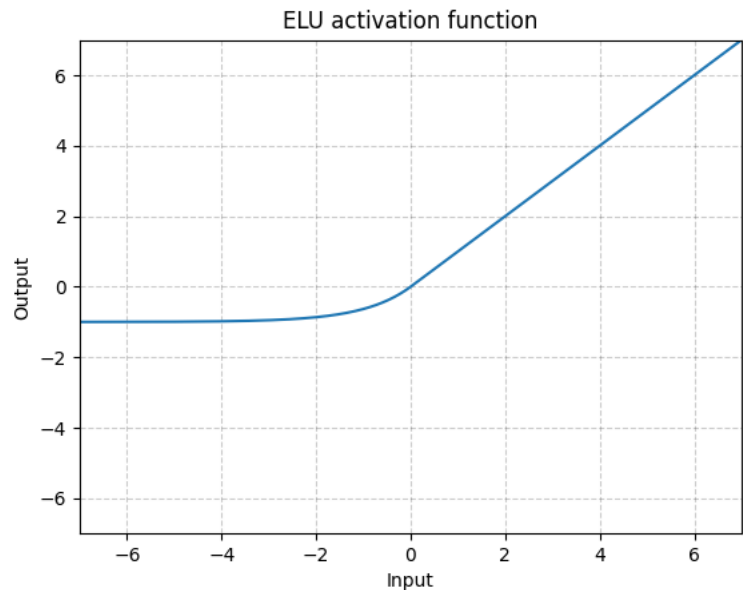
For numerical stability the implementation reverts to the linear function when $input \times \beta > threshold$.



ELU – `nn.ELU()`

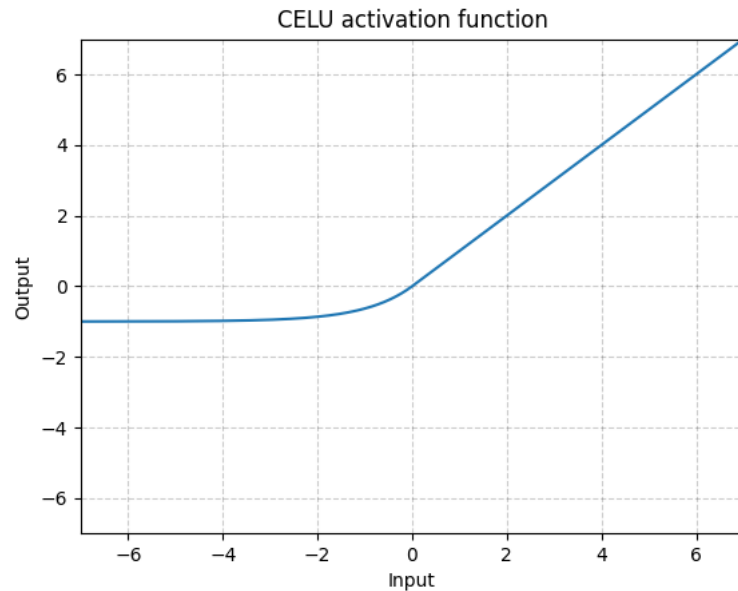
Applies the element-wise function:

$$\text{ELU}(x) = \max(0, x) + \min(0, \alpha * (\exp(x) - 1))$$



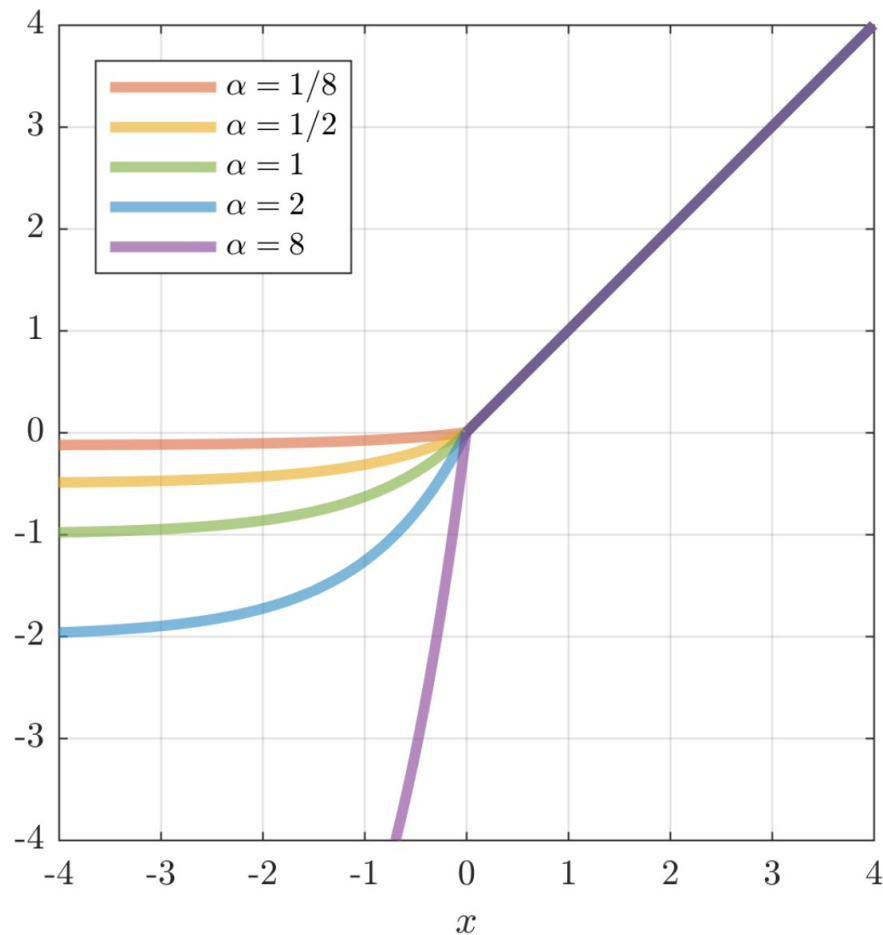
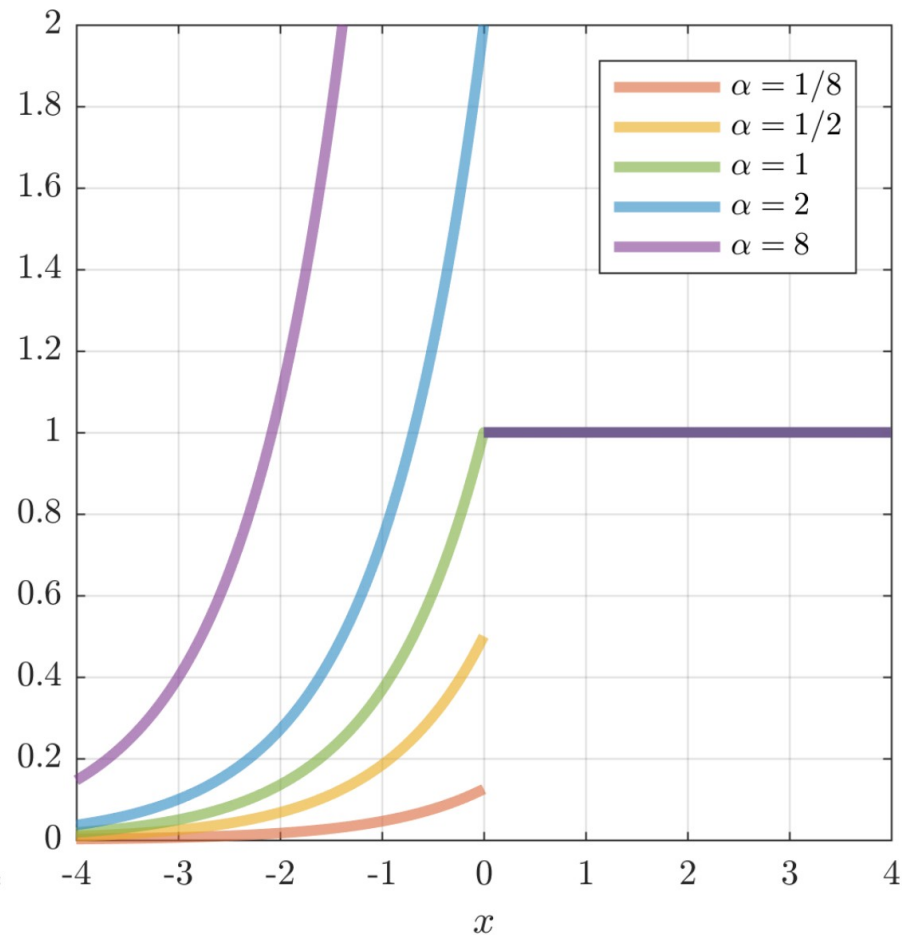
CELU – `nn.CELU()`

$$\text{CELU}(x) = \max(0, x) + \min(0, \alpha * (\exp(x/\alpha) - 1))$$

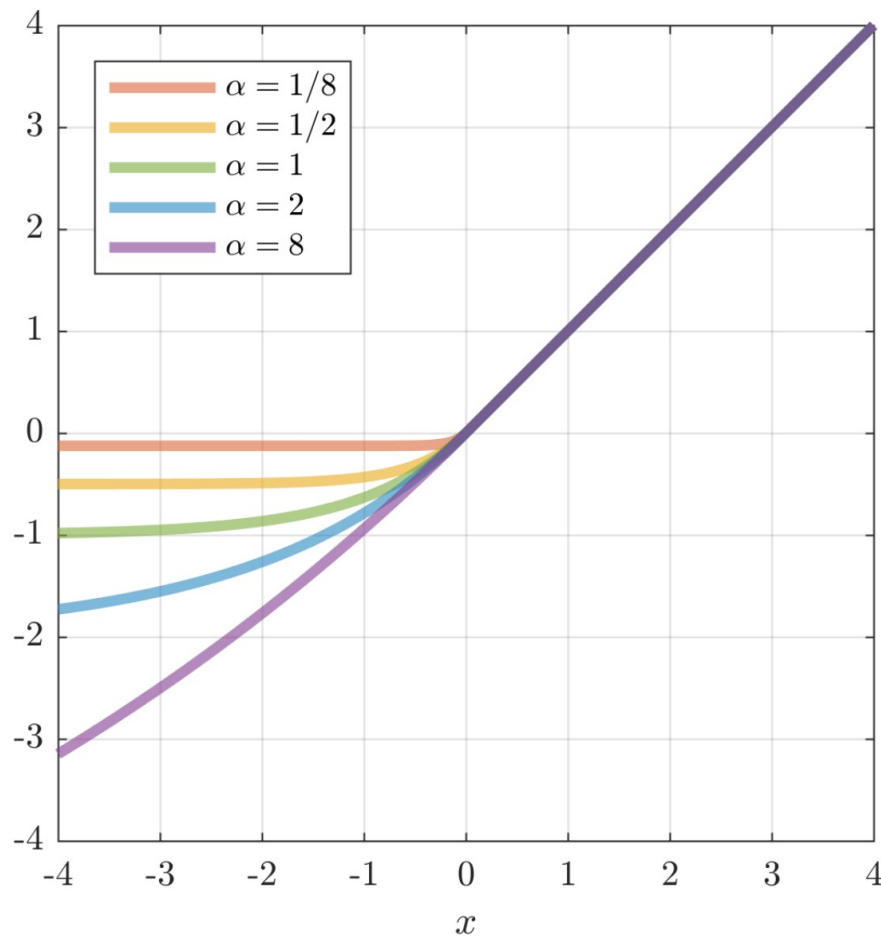
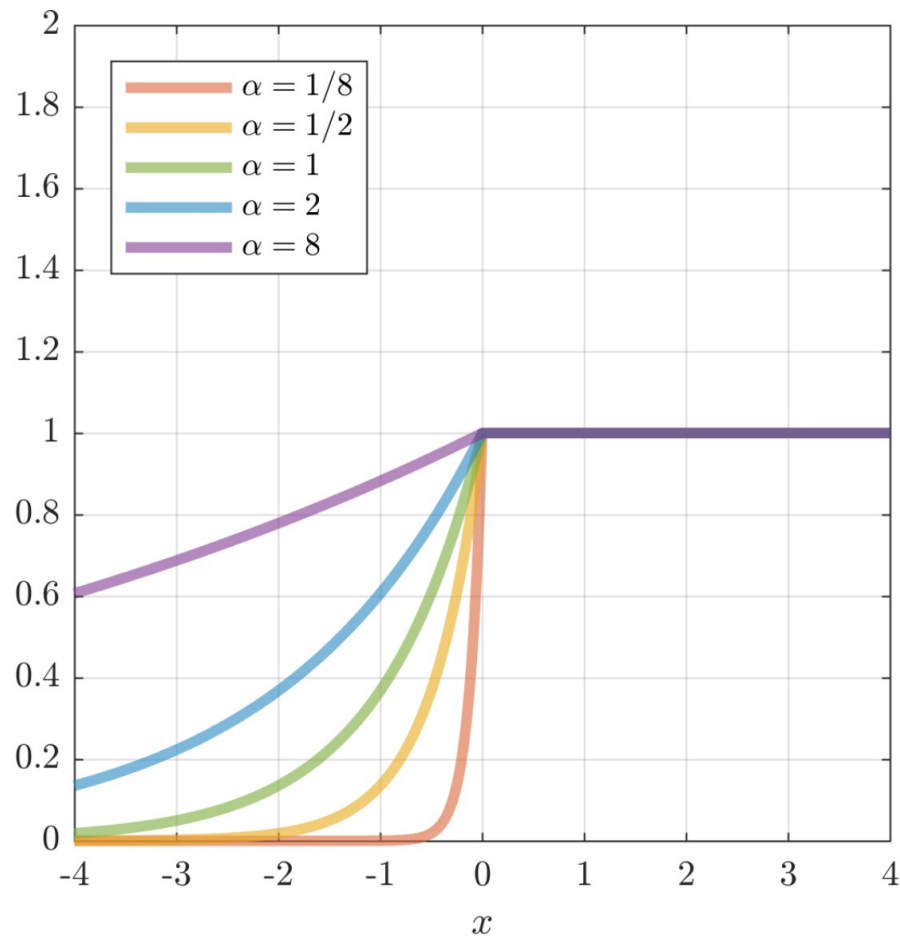


Barron (2017) Continuously differentiable exponential linear units

CELU – nn.CELU()

(a) $\text{ELU}(x, \alpha)$ (b) $\frac{d}{dx} \text{ELU}(x, \alpha)$

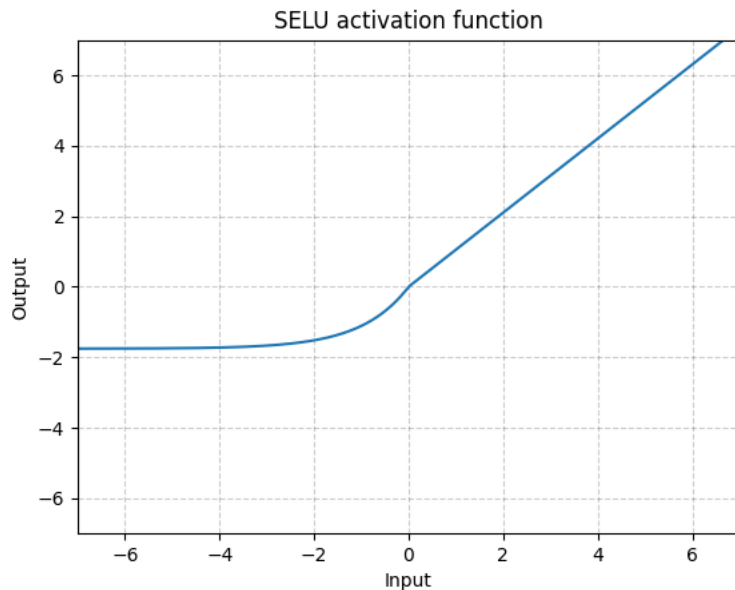
CELU – `nn.CELU()`

(c) $\text{CELU}(x, \alpha)$ (d) $\frac{d}{dx} \text{CELU}(x, \alpha)$

SELU – `nn.SELU()`

$$\text{SELU}(x) = \text{scale} * (\max(0, x) + \min(0, \alpha * (\exp(x) - 1)))$$

with $\alpha = 1.6732632423543772848170429916717$ and $\text{scale} = 1.0507009873554804934193349852946$.



SELU – nn.SELU()

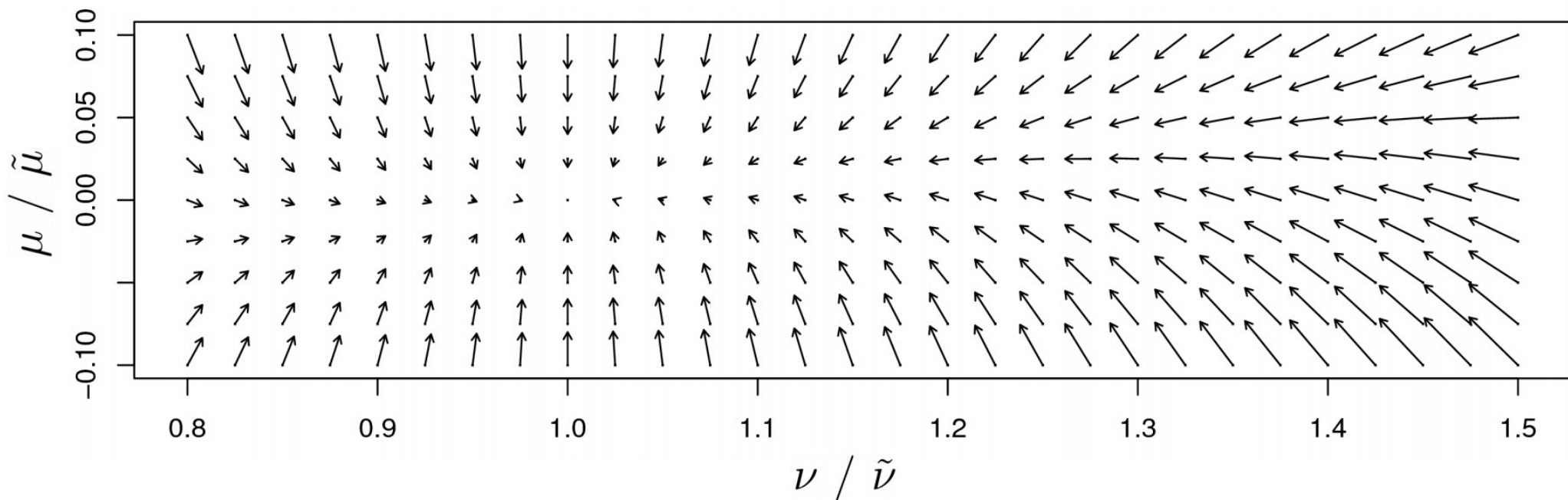
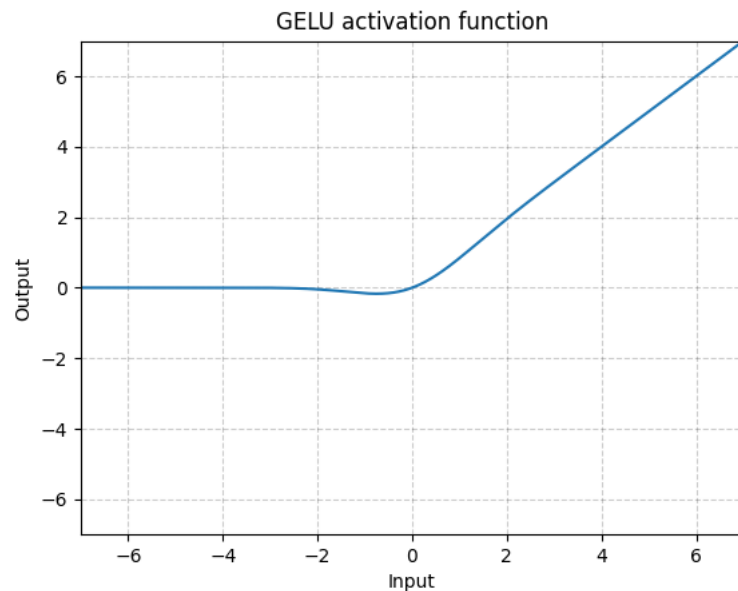


Figure 2: For $\omega = 0$ and $\tau = 1$, the mapping g of mean μ (x -axis) and variance ν (y -axis) to the next layer's mean $\tilde{\mu}$ and variance $\tilde{\nu}$ is depicted. Arrows show in which direction (μ, ν) is mapped by $g : (\mu, \nu) \mapsto (\tilde{\mu}, \tilde{\nu})$. The fixed point of the mapping g is $(1, 1)$.

GELU – `nn.GELU()`

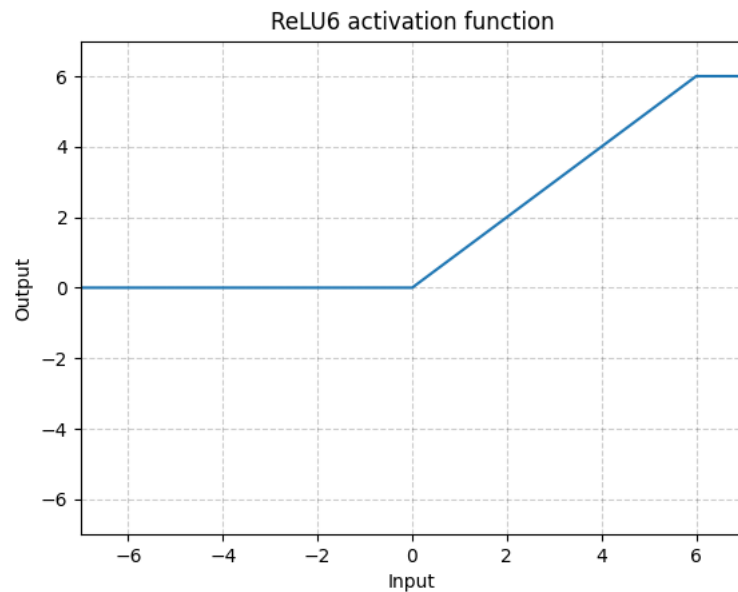
$$\text{GELU}(x) = x * \Phi(x)$$

where $\Phi(x)$ is the Cumulative Distribution Function for Gaussian Distribution.



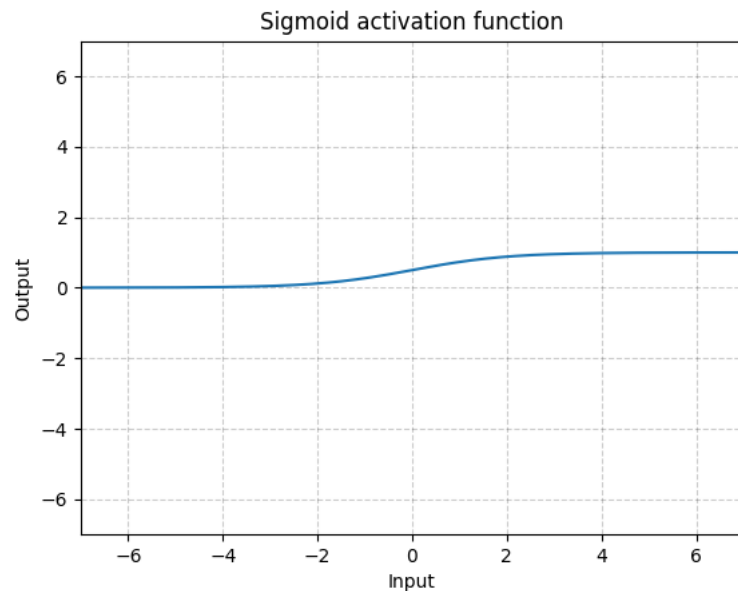
ReLU6 – `nn.ReLU6()`

$$\text{ReLU6}(x) = \min(\max(0, x), 6)$$



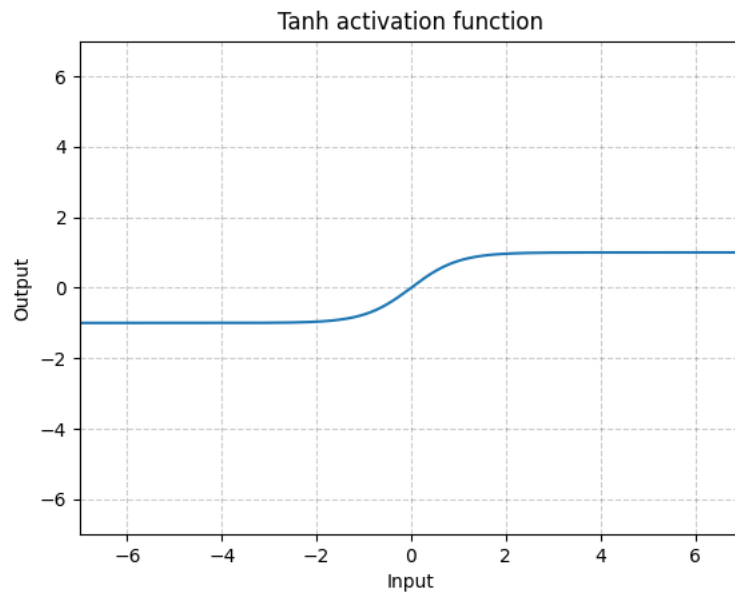
Sigmoid – `nn.Sigmoid()`

$$\text{Sigmoid}(x) = \sigma(x) = \frac{1}{1 + \exp(-x)}$$



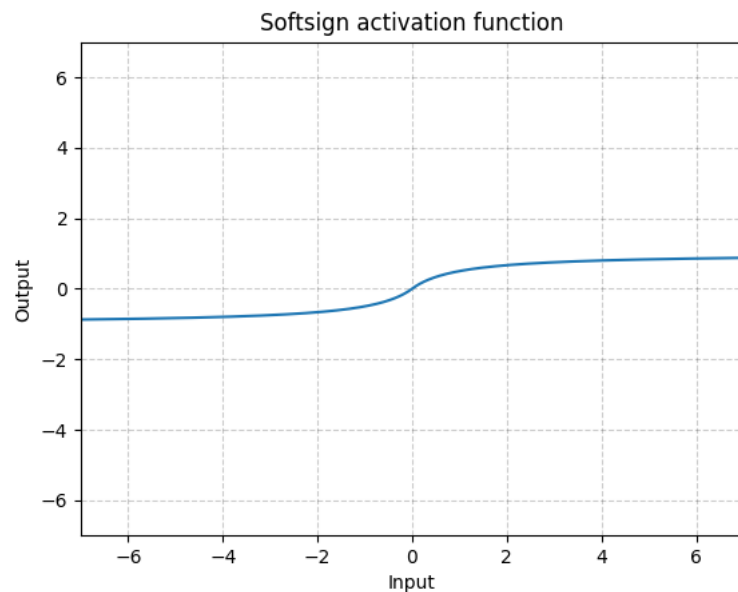
Tanh – `nn.Tanh()`

$$\text{Tanh}(x) = \tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



Softsign – `nn.Softsign()`

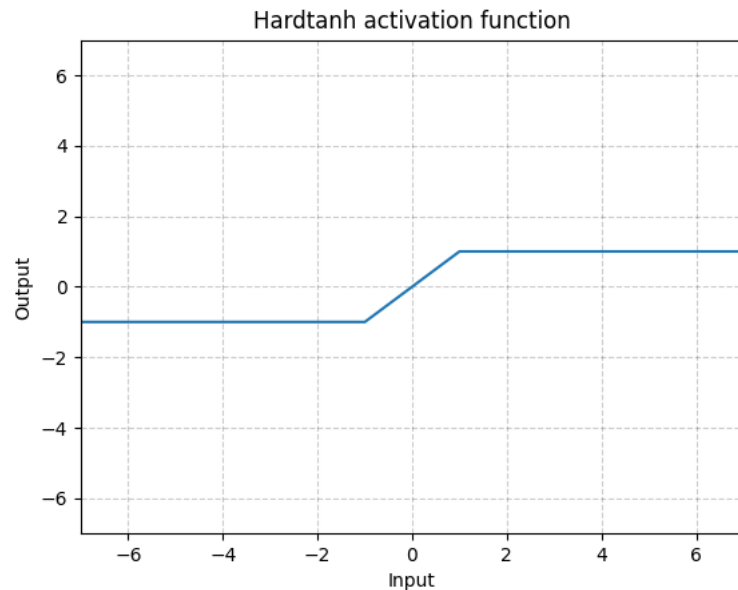
$$\text{SoftSign}(x) = \frac{x}{1 + |x|}$$



Hardtanh – `nn.Hardtanh()`

$$\text{HardTanh}(x) = \begin{cases} 1 & \text{if } x > 1 \\ -1 & \text{if } x < -1 \\ x & \text{otherwise} \end{cases}$$

The range of the linear region $[-1, 1]$ can be adjusted using `min_val` and `max_val`.



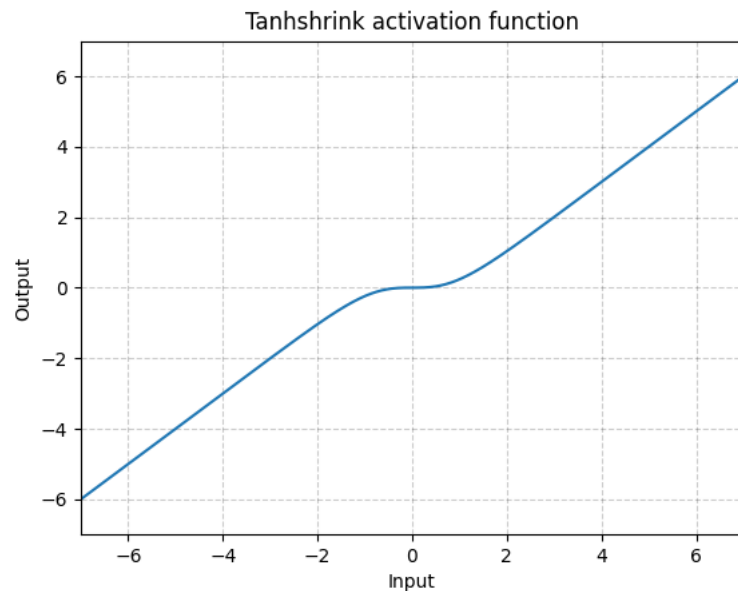
Threshold – `nn.Threshold()`

Threshold is defined as:

$$y = \begin{cases} x, & \text{if } x > \text{threshold} \\ \text{value}, & \text{otherwise} \end{cases}$$

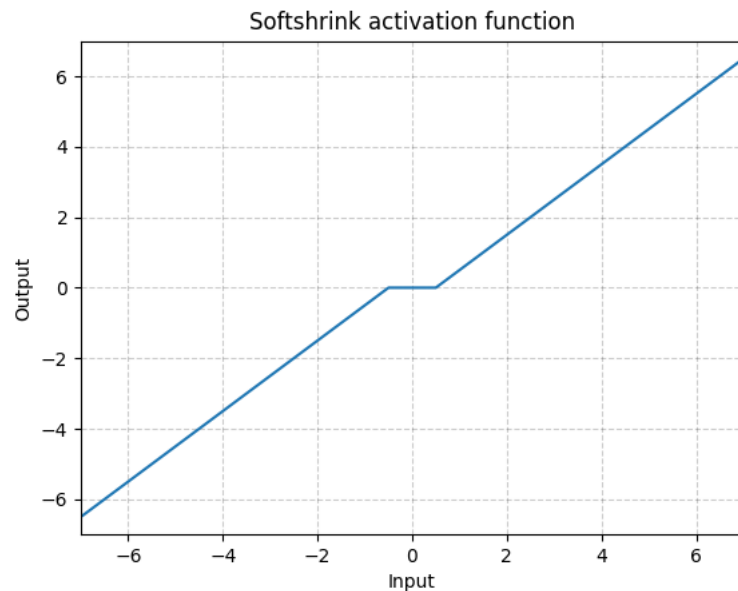
Tanhshrink – `nn.Tanhshrink()`

$$\text{Tanhshrink}(x) = x - \tanh(x)$$



Softshrink – `nn.Softshrink()`

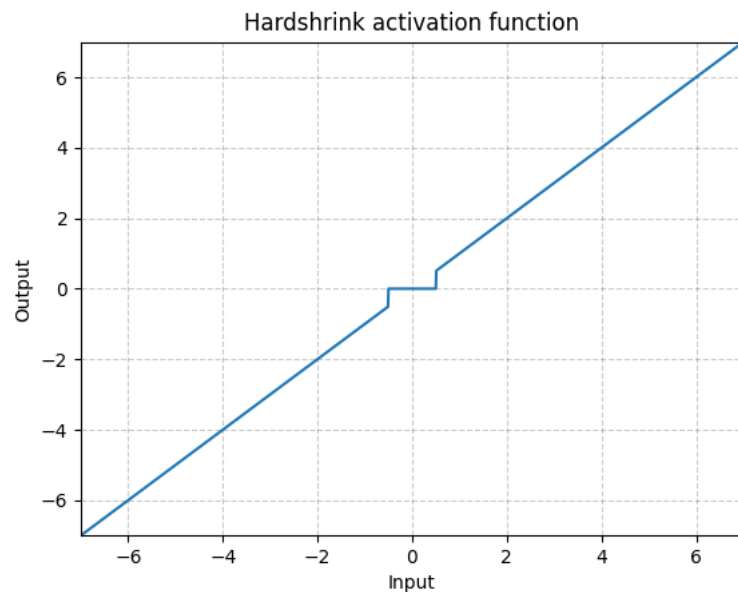
$$\text{SoftShrinkage}(x) = \begin{cases} x - \lambda, & \text{if } x > \lambda \\ x + \lambda, & \text{if } x < -\lambda \\ 0, & \text{otherwise} \end{cases}$$



Hardshrink – `nn.Hardshrink()`

Applies the hard shrinkage function element-wise:

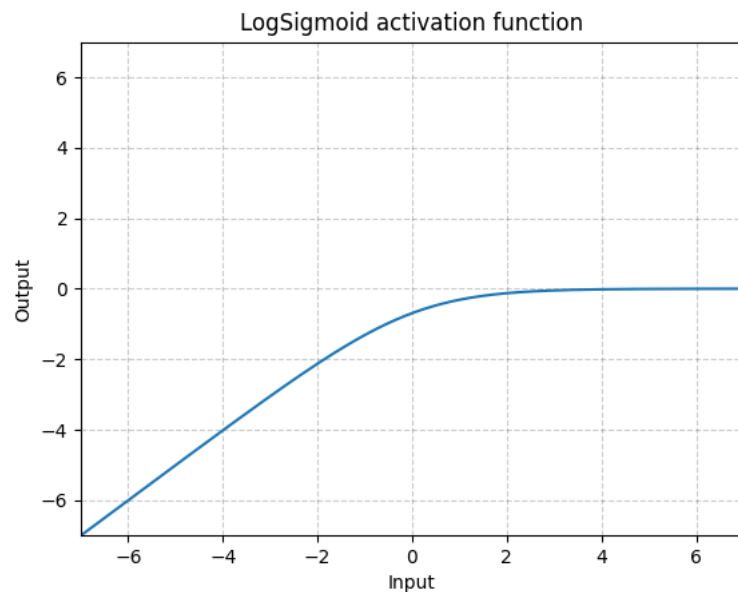
$$\text{HardShrink}(x) = \begin{cases} x, & \text{if } x > \lambda \\ x, & \text{if } x < -\lambda \\ 0, & \text{otherwise} \end{cases}$$



LogSigmoid – `nn.LogSigmoid()`

Applies the element-wise function:

$$\text{LogSigmoid}(x) = \log \left(\frac{1}{1 + \exp(-x)} \right)$$



Softmin – `nn.Softmin()`

Applies the Softmin function to an n-dimensional input Tensor rescaling them so that the elements of the n-dimensional output Tensor lie in the range $[0, 1]$ and sum to 1.

Softmin is defined as:

$$\text{Softmin}(x_i) = \frac{\exp(-x_i)}{\sum_j \exp(-x_j)}$$

Softmax – `nn.Softmax()`

Applies the Softmax function to an n-dimensional input Tensor rescaling them so that the elements of the n-dimensional output Tensor lie in the range $[0,1]$ and sum to 1.

Softmax is defined as:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

LogSoftmax – `nn.LogSoftmax()`

Applies the $\log(\text{Softmax}(x))$ function to an n-dimensional input Tensor. The LogSoftmax formulation can be simplified as:

$$\text{LogSoftmax}(x_i) = \log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right)$$

Cost Functions

In PyTorch & how to use them



MSE – `nn.MSELoss()`

Creates a criterion that measures the mean squared error (squared L2 norm) between each element in the input x and target y .

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = (x_n - y_n)^2,$$

where N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then:

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction} = \text{'mean'}; \\ \text{sum}(L), & \text{if reduction} = \text{'sum'}. \end{cases}$$

x and y are tensors of arbitrary shapes with a total of n elements each.

The sum operation still operates over all the elements, and divides by n .

The division by n can be avoided if one sets `reduction = 'sum'`.

L1 – nn.L1Loss()

Creates a criterion that measures the mean absolute error (MAE) between each element in the input x and target y .

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = |x_n - y_n|,$$

where N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then:

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction} = \text{'mean'}; \\ \text{sum}(L), & \text{if reduction} = \text{'sum'}. \end{cases}$$

x and y are tensors of arbitrary shapes with a total of n elements each.

The sum operation still operates over all the elements, and divides by n .

The division by n can be avoided if one sets `reduction = 'sum'`.

nn.SmoothL1Loss()

Creates a criterion that uses a squared term if the absolute element-wise error falls below 1 and an L1 term otherwise. It is less sensitive to outliers than the *MSELoss* and in some cases prevents exploding gradients (e.g. see *Fast R-CNN* paper by Ross Girshick). Also known as the Huber loss:

$$\text{loss}(x, y) = \frac{1}{n} \sum_i z_i$$

where z_i is given by:

$$z_i = \begin{cases} 0.5(x_i - y_i)^2, & \text{if } |x_i - y_i| < 1 \\ |x_i - y_i| - 0.5, & \text{otherwise} \end{cases}$$

x and y arbitrary shapes with a total of n elements each the sum operation still operates over all the elements, and divides by n .

The division by n can be avoided if sets `reduction = 'sum'`.

NLL – nn.NLLLoss()

The negative log likelihood loss. It is useful to train a classification problem with C classes.

If provided, the optional argument `weight` should be a 1D Tensor assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set.

The *target* that this loss expects should be a class index in the range $[0, C - 1]$ where $C = \text{number of classes}$; if *ignore_index* is specified, this loss also accepts this class index (this index may not necessarily be in the class range).

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_{y_n} x_{n, y_n}, \quad w_c = \text{weight}[c] \cdot 1\{c \neq \text{ignore_index}\},$$

where x is the input, y is the target, w is the weight, and N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then

$$\ell(x, y) = \begin{cases} \sum_{n=1}^N \frac{1}{\sum_{n=1}^N w_{y_n}} l_n, & \text{if reduction = 'mean'}; \\ \sum_{n=1}^N l_n, & \text{if reduction = 'sum'}. \end{cases}$$

CE – `nn.CrossEntropyLoss()`

This criterion combines `nn.LogSoftmax()` and `nn.NLLLoss()` in one single class.

It is useful when training a classification problem with C classes. If provided, the optional argument `weight` should be a 1D *Tensor* assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set.

The *input* is expected to contain raw, unnormalized scores for each class.

input has to be a *Tensor* of size either $(minibatch, C)$ or $(minibatch, C, d_1, d_2, \dots, d_K)$ with $K \geq 1$ for the K -dimensional case (described later).

This criterion expects a class index in the range $[0, C - 1]$ as the *target* for each value of a 1D *tensor* of size *minibatch*; if *ignore_index* is specified, this criterion also accepts this class index (this index may not necessarily be in the class range).

CE – `nn.CrossEntropyLoss()`

The loss can be described as:

$$\text{loss}(x, \text{class}) = -\log \left(\frac{\exp(x[\text{class}])}{\sum_j \exp(x[j])} \right) = -x[\text{class}] + \log \left(\sum_j \exp(x[j]) \right)$$

or in the case of the `weight` argument being specified:

$$\text{loss}(x, \text{class}) = \text{weight}[\text{class}] \left(-x[\text{class}] + \log \left(\sum_j \exp(x[j]) \right) \right)$$

The losses are averaged across observations for each minibatch.

Can also be used for higher dimension inputs, such as 2D images, by providing an input of size $(\text{minibatch}, C, d_1, d_2, \dots, d_K)$ with $K \geq 1$, where K is the number of dimensions, and a target of appropriate shape (see below).

nn.AdaptiveLogSoftmaxWithLoss()

Efficient softmax approximation as described in [Efficient softmax approximation for GPUs](#) by Edouard Grave, Armand Joulin, Moustapha Cissé, David Grangier, and Hervé Jégou.

Adaptive softmax is an approximate strategy for training models with large output spaces. It is most effective when the label distribution is highly imbalanced, for example in natural language modelling, where the word frequency distribution approximately follows the [Zipf's law](#).

Adaptive softmax partitions the labels into several clusters, according to their frequency. These clusters may contain different number of targets each. Additionally, clusters containing less frequent labels assign lower dimensional embeddings to those labels, which speeds up the computation. For each minibatch, only clusters for which at least one target is present are evaluated.

The idea is that the clusters which are accessed frequently (like the first one, containing most frequent labels), should also be cheap to compute – that is, contain a small number of assigned labels.

KLDiv – nn.KLDivLoss()

KL divergence is a useful distance measure for continuous distributions and is often useful when performing direct regression over the space of (discretely sampled) continuous output distributions.

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$l(x, y) = L = \{l_1, \dots, l_N\}, \quad l_n = y_n \cdot (\log y_n - x_n)$$

where the index N spans all dimensions of `input` and L has the same shape as `input`. If `reduction` is not `'none'` (default `'mean'`), then:

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction} = \text{'mean'}; \\ \text{sum}(L), & \text{if reduction} = \text{'sum'}. \end{cases}$$

In default `reduction` mode `'mean'`, the losses are averaged for each minibatch over observations **as well as** over dimensions. `'batchmean'` mode gives the correct KL divergence where losses are averaged over batch dimension only. `'mean'` mode's behavior will be changed to the same as `'batchmean'` in the next major release.

Poisson – `nn.PoissonNLLLoss()`

Negative log likelihood loss with Poisson distribution of target.

The loss can be described as:

$$\text{target} \sim \text{Poisson}(\text{input}) \text{loss}(\text{input}, \text{target}) = \text{input} - \text{target} * \log(\text{input}) + \log(\text{target}!)$$

The last term can be omitted or approximated with Stirling formula. The approximation is used for target values more than 1. For targets less or equal to 1 zeros are added to the loss.

BCE – `nn.BCELoss()`

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_n [y_n \cdot \log x_n + (1 - y_n) \cdot \log(1 - x_n)],$$

where N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction} = \text{'mean'}; \\ \text{sum}(L), & \text{if reduction} = \text{'sum'}. \end{cases}$$

This is used for measuring the error of a reconstruction in for example an auto-encoder. Note that the targets y should be numbers between 0 and 1.

BCE – `nn.BCELoss()`

Notice that if x_n is either 0 or 1, one of the log terms would be mathematically undefined in the above loss equation. PyTorch chooses to set $\log(0) = -\infty$, since $\lim_{x \rightarrow 0} \log(x) = -\infty$. However, an infinite term in the loss equation is not desirable for several reasons.

For one, if either $y_n = 0$ or $(1 - y_n) = 0$, then we would be multiplying 0 with infinity. Secondly, if we have an infinite loss value, then we would also have an infinite term in our gradient, since $\lim_{x \rightarrow 0} \frac{d}{dx} \log(x) = \infty$. This would make BCELoss's backward method nonlinear with respect to x_n , and using it for things like linear regression would not be straight-forward.

Our solution is that BCELoss clamps its log function outputs to be greater than or equal to -100. This way, we can always have a finite loss value and a linear backward method.

BCE – `nn.BCEWithLogitsLoss()`

This loss combines a *Sigmoid* layer and the *BCELoss* in one single class. This version is more numerically stable than using a plain *Sigmoid* followed by a *BCELoss* as, by combining the operations into one layer, we take advantage of the log-sum-exp trick for numerical stability.

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_n [y_n \cdot \log \sigma(x_n) + (1 - y_n) \cdot \log(1 - \sigma(x_n))],$$

where N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction} = \text{'mean'}; \\ \text{sum}(L), & \text{if reduction} = \text{'sum'}. \end{cases}$$

This is used for measuring the error of a reconstruction in for example an auto-encoder. Note that the targets $t[i]$ should be numbers between 0 and 1.

BCE – `nn.BCEWithLogitsLoss()`

It's possible to trade off recall and precision by adding weights to positive examples. In the case of multi-label classification the loss can be described as:

$$\ell_c(x, y) = L_c = \{l_{1,c}, \dots, l_{N,c}\}^\top, \quad l_{n,c} = -w_{n,c} [p_c y_{n,c} \cdot \log \sigma(x_{n,c}) + (1 - y_{n,c}) \cdot \log(1 - \sigma(x_{n,c}))],$$

where c is the class number ($c > 1$ for multi-label binary classification, $c = 1$ for single-label binary classification), n is the number of the sample in the batch and p_c is the weight of the positive answer for the class c .

$p_c > 1$ increases the recall, $p_c < 1$ increases the precision.

For example, if a dataset contains 100 positive and 300 negative examples of a single class, then *pos_weight* for the class should be equal to $\frac{300}{100} = 3$. The loss would act as if the dataset contains $3 \times 100 = 300$ positive examples.

Ranking – `nn.MarginRankingLoss()`

Creates a criterion that measures the loss given inputs x_1 , x_2 , two 1D mini-batch *Tensors*, and a label 1D mini-batch tensor y (containing 1 or -1).

If $y = 1$ then it assumed the first input should be ranked higher (have a larger value) than the second input, and vice-versa for $y = -1$.

The loss function for each sample in the mini-batch is:

$$\text{loss}(x, y) = \max(0, -y * (x_1 - x_2) + \text{margin})$$

nn.TripletMarginLoss()

Creates a criterion that measures the triplet loss given an input tensors x_1 , x_2 , x_3 and a margin with a value greater than 0. This is used for measuring a relative similarity between samples. A triplet is composed by a , p and n (i.e., *anchor*, *positive examples* and *negative examples* respectively). The shapes of all input tensors should be (N, D) .

The distance swap is described in detail in the paper [Learning shallow convolutional feature descriptors with triplet losses](#) by V. Balntas, E. Riba et al.

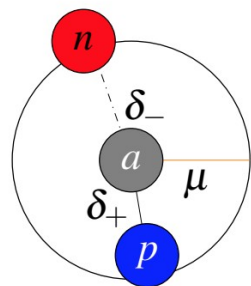
The loss function for each sample in the mini-batch is:

$$L(a, p, n) = \max\{d(a_i, p_i) - d(a_i, n_i) + \text{margin}, 0\}$$

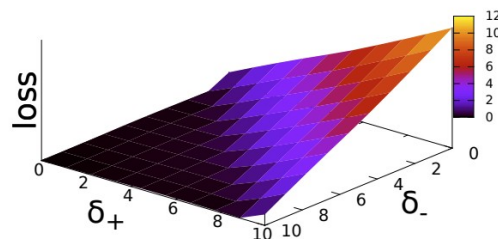
where

$$d(x_i, y_i) = \|\mathbf{x}_i - \mathbf{y}_i\|_p$$

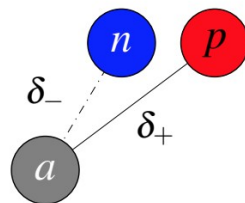
nn.TripletMarginLoss()



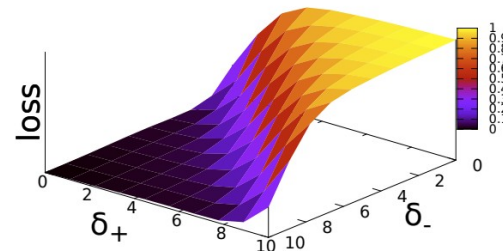
(a)



(b)



(c)



(d)

Figure 1: (a) Margin ranking loss. It seeks to push n outside the circle defined by the margin μ , and pull p inside. (b) Margin ranking loss values in function of δ_- , δ_+ (c) Ratio loss. It seeks to force δ_+ to be much smaller than δ_- . (d) Ratio loss values in function of δ_- , δ_+

$$\hat{\delta}_+ = ||f(\mathbf{a}) - f(\mathbf{p})||_2 \text{ and } \hat{\delta}_- = ||\hat{f}(\mathbf{a}) - f(\mathbf{n})||_2.$$

$$\lambda(\delta_+, \hat{\delta}_*) = \max(0, \mu + \delta_+ - \hat{\delta}_*).$$

`nn.SoftMarginLoss()`

Creates a criterion that optimizes a two-class classification logistic loss between input tensor x and target tensor y (containing 1 or -1).

$$\text{loss}(x, y) = \sum_i \frac{\log(1 + \exp(-y[i] * x[i]))}{x.\text{nelement}()}$$

nn.MultiLabelMarginLoss()

Creates a criterion that optimizes a multi-class multi-classification hinge loss (margin-based loss) between input x (a 2D mini-batch *Tensor*) and output y (which is a 2D *Tensor* of target class indices). For each sample in the mini-batch:

$$\text{loss}(x, y) = \sum_{ij} \frac{\max(0, 1 - (x[y[j]] - x[i]))}{x.size(0)}$$

where $x \in \{0, \dots, x.size(0) - 1\}$, $y \in \{0, \dots, y.size(0) - 1\}$, $0 \leq y[j] \leq x.size(0) - 1$, and $i \neq y[j]$ for all i and j .

y and x must have the same size.

The criterion only considers a contiguous block of non-negative targets that starts at the front.

This allows for different samples to have variable amounts of target classes.

nn.MultiLabelSoftMarginLoss()

Creates a criterion that optimizes a multi-label one-versus-all loss based on max-entropy, between input x and target y of size (N, C) . For each sample in the minibatch:

$$loss(x, y) = -\frac{1}{C} * \sum_i y[i] * \log((1 + \exp(-x[i]))^{-1}) + (1 - y[i]) * \log\left(\frac{\exp(-x[i])}{(1 + \exp(-x[i]))}\right)$$

where $i \in \{0, \dots, x.nElement() - 1\}, y[i] \in \{0, 1\}$.

nn.MultiMarginLoss()

Creates a criterion that optimizes a multi-class classification hinge loss (margin-based loss) between input x (a 2D mini-batch *Tensor*) and output y (which is a 1D tensor of target class indices, $0 \leq y \leq x.size(1) - 1$):

For each mini-batch sample, the loss in terms of the 1D input x and scalar output y is:

$$\text{loss}(x, y) = \frac{\sum_i \max(0, \text{margin} - x[y] + x[i])^p}{x.size(0)}$$

where $x \in \{0, \dots, x.size(0) - 1\}$ and $i \neq y$.

Optionally, you can give non-equal weighting on the classes by passing a 1D `weight` tensor into the constructor.

The loss function then becomes:

$$\text{loss}(x, y) = \frac{\sum_i \max(0, w[y] * (\text{margin} - x[y] + x[i]))^p}{x.size(0)}$$

nn.HingeEmbeddingLoss()

Measures the loss given an input tensor x and a labels tensor y (containing 1 or -1). This is usually used for measuring whether two inputs are similar or dissimilar, e.g. using the L1 pairwise distance as x , and is typically used for learning nonlinear embeddings or semi-supervised learning.

The loss function for n -th sample in the mini-batch is

$$l_n = \begin{cases} x_n, & \text{if } y_n = 1, \\ \max\{0, \Delta - x_n\}, & \text{if } y_n = -1, \end{cases}$$

and the total loss functions is

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction = 'mean'}; \\ \text{sum}(L), & \text{if reduction = 'sum'}. \end{cases}$$

where $L = \{l_1, \dots, l_N\}^\top$.

nn.CosineEmbeddingLoss()

Creates a criterion that measures the loss given input tensors x_1 , x_2 and a *Tensor* label y with values 1 or -1. This is used for measuring whether two inputs are similar or dissimilar, using the cosine distance, and is typically used for learning nonlinear embeddings or semi-supervised learning.

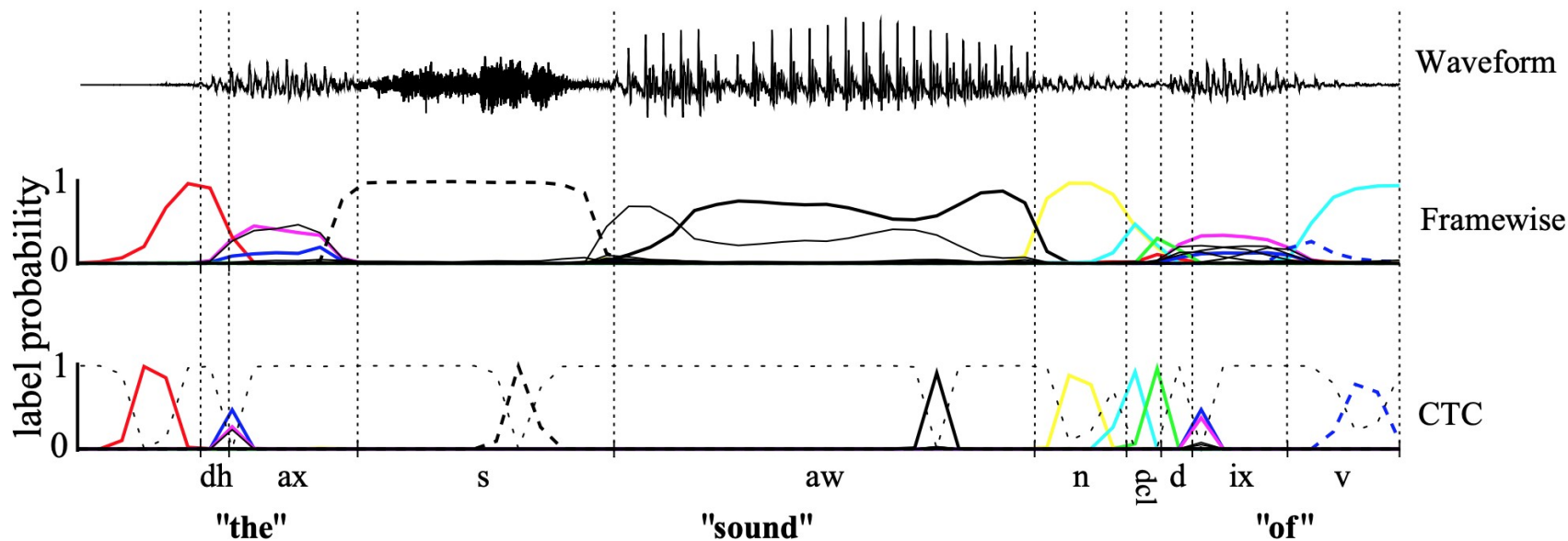
The loss function for each sample is:

$$\text{loss}(x, y) = \begin{cases} 1 - \cos(x_1, x_2), & \text{if } y = 1 \\ \max(0, \cos(x_1, x_2) - \text{margin}), & \text{if } y = -1 \end{cases}$$

CTC – nn.CTCLoss()

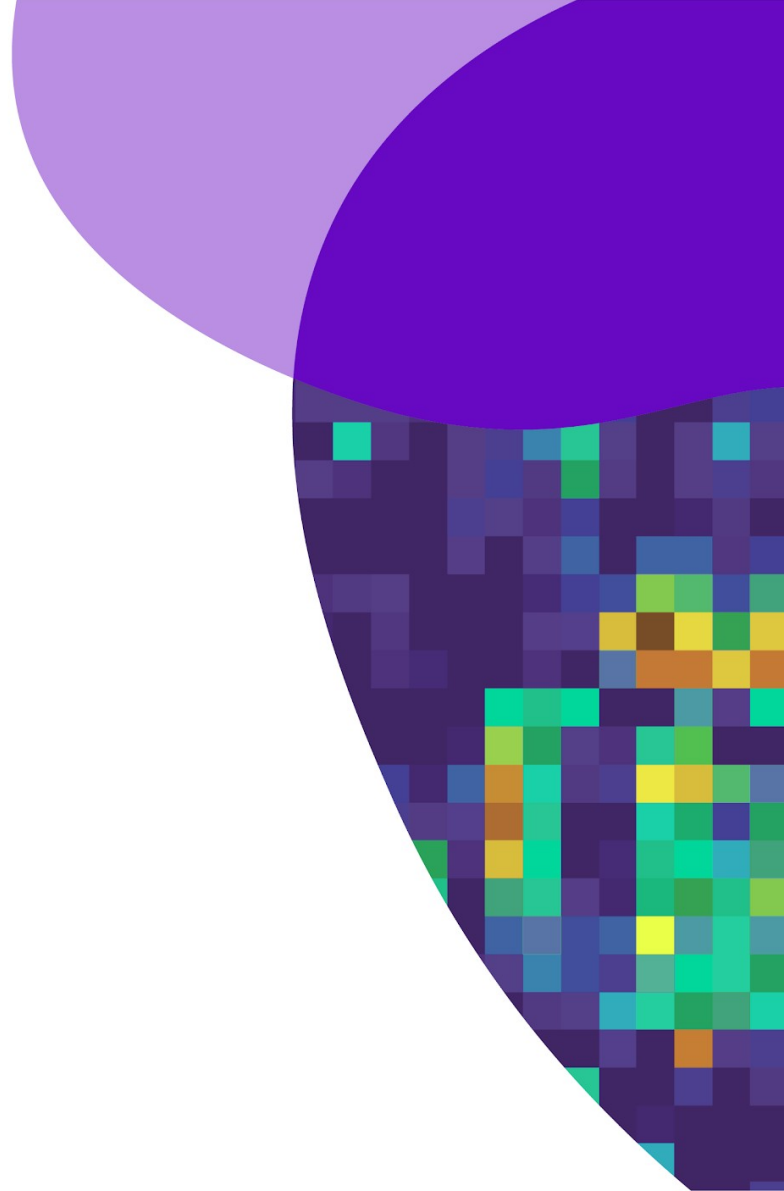
The Connectionist Temporal Classification loss.

Calculates loss between a continuous (unsegmented) time series and a target sequence. CTCLoss sums over the probability of possible alignments of input to target, producing a loss value which is differentiable with respect to each input node. The alignment of input to target is assumed to be “many-to-one”, which limits the length of the target sequence such that it must be $\leq$ the input length.



Architectures

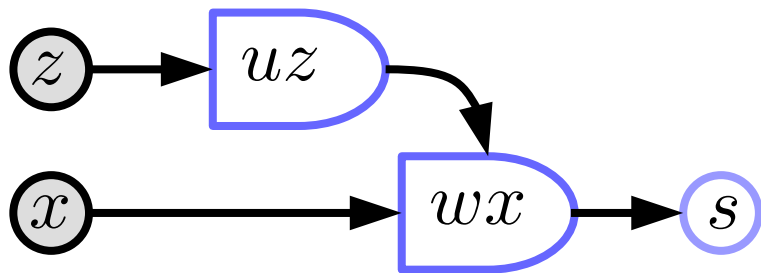
Arranging modules



Multiplicative Modules

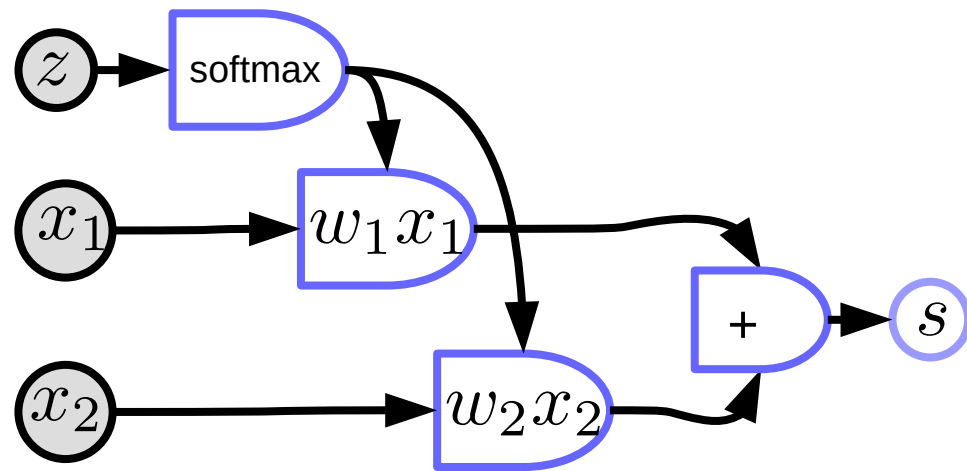
► Quadratic layer, product units, Sigma-Pi units

$$s_i = \sum_j w_{ij} x_j \text{ with } w_{ij} = \sum_k u_{ijk} z_k ; \text{equiv} : s_i = \sum_{jk} u_{ijk} z_k x_j$$



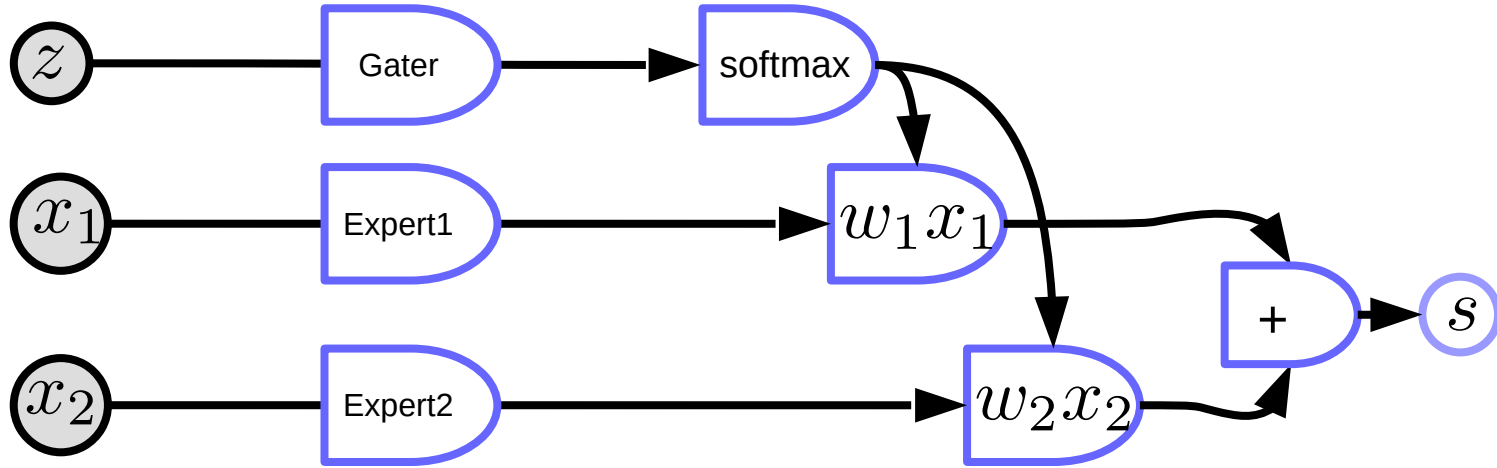
► Attention module

$$s_i = \sum_j w_j x_{ij} \quad w_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$



Mixture of Experts

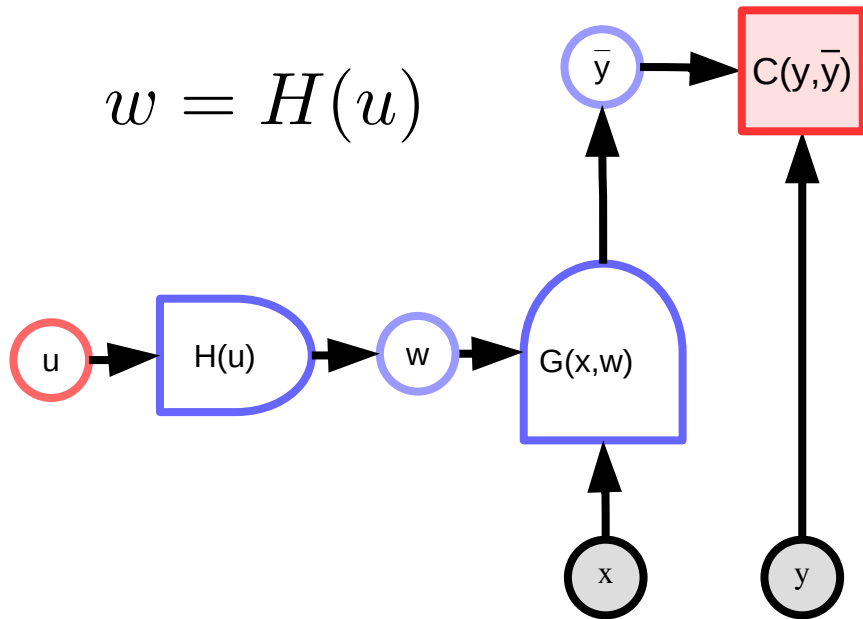
► Attention module “Switches” expert networks



$$s_i = \sum_j w_j x_{ij} \quad w_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Parameter transformations

- ▶ When the parameter vector is the output of a function



$$u \leftarrow u - \eta \frac{\partial H^T}{\partial u} \frac{\partial C^T}{\partial w}$$

$$w \leftarrow w - \eta \frac{\partial H}{\partial u} \frac{\partial H^T}{\partial u} \frac{\partial C^T}{\partial w}$$

$$[N_w \times N_u] \quad [N_u \times N_w] \quad [N_w \times 1]$$

Simple parameter transform: weight sharing

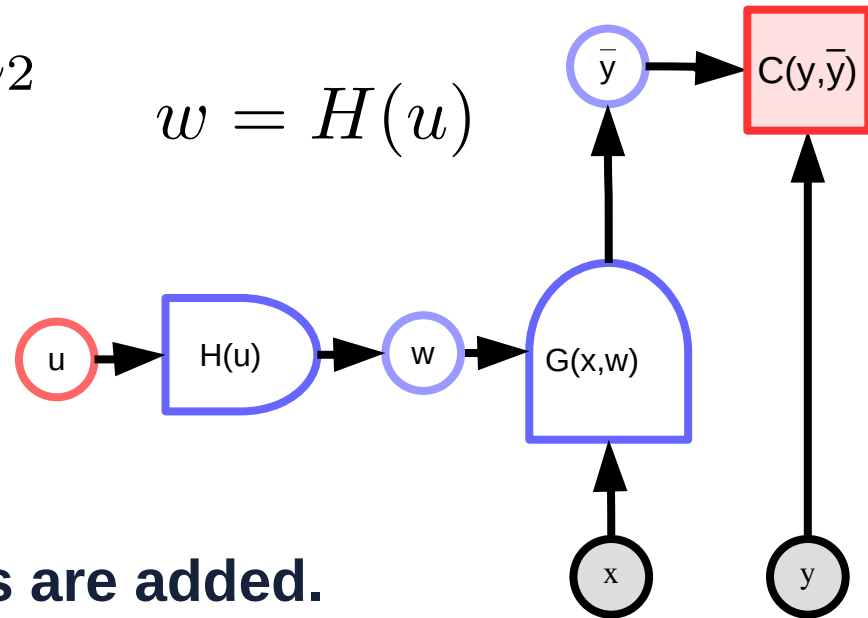
- ▶ Function $H(u)$ replicates one component of u into multiple components of w

$$w_1, w_2 = u_1 \quad w_3 = w_4 = u_2$$

- ▶ H is like a “Y” branch.

- ▶ Gradients are summed in the backprop

- ▶ The gradients w.r.t. shared parameters are added.



Shared Weights for Motif Detection

- Detecting motifs anywhere on an input

